

Auditoria Técnica

Gestor Financeiro Pessoal — index.html

Do protótipo local ao produto vendável, com um agente de IA financeiro

Preparado para Nico · 25 de agosto de 2026

1. Resumo Executivo

Este documento é uma auditoria completa ao ficheiro index.html enviado — uma aplicação de gestão financeira pessoal construída em HTML, Bootstrap, FullCalendar e Chart.js, com todos os dados guardados no localStorage do browser. A base de trabalho é sólida para um protótipo: a interface é bonita, cobre as categorias certas (renda, dívidas, essenciais, lazer, poupança, investimentos, cofre de emergência) e já tem calendário, gráficos e exportação CSV.

No entanto, tal como está, o projeto não está pronto para ser usado como gestor financeiro "a sério" — e está ainda mais longe de poder ser vendido a terceiros ou de servir de base a um agente de IA. Foram encontrados bugs de cálculo reais (não só cosméticos), um problema sério de perda de dados (tudo vive apenas num browser, sem cópia de segurança), uma vulnerabilidade de segurança (XSS), e uma limitação estrutural mais profunda: o modelo de dados mistura três conceitos diferentes — contas, categorias de despesa e metas — num único campo "type", o que vai continuar a gerar bugs e limitações à medida que o produto cresce.

A boa notícia é que nada disto é difícil de corrigir, e a estrutura de dados atual (uma lista de eventos com data, título, valor e tipo) é um ponto de partida perfeitamente aproveitável. A secção 8 propõe um caminho em 4 fases: (1) estabilizar e proteger os dados agora, sem mudar de tecnologia; (2) redesenhar o modelo de dados e passar para uma arquitetura real com conta de utilizador e base de dados; (3) preparar o produto para ser vendido a outras pessoas; (4) só depois disso, ligar um agente de IA — porque um agente de IA só é tão bom quanto os dados que lhe são dados a analisar.

2. Pontos Fortes do Protótipo Atual

Vale destacar o que já está bem pensado, porque é a base sobre a qual vamos construir:

- Cobertura de categorias completa e relevante para uma vida financeira real: renda, dívidas, despesas essenciais, lazer, despesas fixas, poupança, investimento e cofre de emergência.
- Visualização forte: cartões de indicadores, calendário mensal, 5 gráficos diferentes (distribuição, entradas/saídas, agendamento de dívidas, crescimento de investimento, resumo mensal) e modais de detalhe por categoria.
- Funcionalidades já pensadas para o dia a dia: eventos recorrentes (diário/semanal/mensal/anual), parcelamento de dívidas, marcação de dívidas como pagas, edição e remoção (individual ou em série), filtros e exportação para CSV.
- Cuidado com acessibilidade básica: labels em quase todos os campos e botões, atributos aria-label e uso de "visually-hidden" nos ícones.
- O modelo de dados de cada evento (data, título, valor, tipo, estado) é simples e já é 90% do caminho para um esquema de base de dados real.

3. Problemas Críticos de Dados e Segurança

Estes são os problemas que, na minha opinião, devem ser resolvidos antes de confiar dados financeiros reais a esta aplicação — independentemente de decidir vender o produto ou usá-lo só para si.

Problema	Onde	Impacto / Cenário	Severidade
Toda a informação financeira vive só no localStorage de um browser, sem exportação/importação de segurança nem sincronização.	STORAGE_KEY / SETTINGS_KEY, início do <script>	Limpar dados de navegação, trocar de computador, usar modo privado, reinstalar o browser ou uma falha do disco apaga o histórico financeiro todo, de forma irreversível. Existe exportação para CSV, mas não existe importação — ou seja, mesmo com o CSV, não há forma de "restaurar" os dados na aplicação.	Crítica
Inserção de texto do utilizador diretamente em innerHTML sem escapar (XSS).	Linhas ~1130, ~1297, ~1362 e ~1465 — tabelas de próximos eventos, saldo, renda e investimentos	Se o "título" de um evento contiver HTML/JS (por exemplo colado de um extrato bancário ou de um SMS), esse código é executado como script real dentro da aplicação. Hoje o risco é baixo (só afeta o próprio utilizador), mas se no futuro os dados forem partilhados, sincronizados na nuvem ou importados de um ficheiro externo (extrato bancário, por	Alta

Problema	Onde	Impacto / Cenário	Severidade
		exemplo), isto passa a ser uma vulnerabilidade séria entre utilizadores ou dispositivos.	
Nenhum tratamento de erro ao ler o localStorage.	JSON.parse(localStorage.getItem(STORAGE_KEY) '[]')	Se o conteúdo guardado ficar corrompido por qualquer motivo (extensão do browser, falha a meio de uma escrita, etc.), o JSON.parse rebenta e a aplicação deixa de carregar por completo — sem mensagem de erro amigável nem forma de recuperar.	Alta
IDs de eventos gerados com event_\${index}_\${Date.now()}.	Criação de eventos, recorrências	Não é um identificador robusto: depende da posição no array e do relógio do sistema. É suficiente hoje, mas é frágil para uma futura sincronização entre dispositivos ou base de dados (onde é preciso um ID verdadeiramente único, tipo UUID).	Média
Sem qualquer autenticação, PIN ou bloqueio de acesso à aplicação.	Aplicação inteira	Qualquer pessoa com acesso ao computador/browser vê saldo,	Média

Problema	Onde	Impacto / Cenário	Severidade
		dívidas, investimentos e histórico completo, em texto simples, sem cópia de segurança nem palavra-passe.	

4. Bugs de Cálculo Encontrados

Estes não são detalhes cosméticos — afetam diretamente os números que a aplicação mostra.

Problema	Onde	Impacto / Cenário	Severidade
O "Cofre de Emergência" soma sempre o valor absoluto de cada movimento.	<code>reduce((a,b) => a + (b.amount>0 ? b.amount : -b.amount), 0)</code> — usado 3x (indicadores, gráfico de distribuição, modal do cofre)	A intenção parece ser permitir levantamentos com valor negativo, mas o cálculo transforma qualquer valor negativo em positivo antes de somar. Resultado: um levantamento do cofre de emergência faz o saldo do cofre subir em vez de descer.	Alta
Validação de parcelas de dívida usa igualdade exata entre números decimais.	<code>if (installment * occ !== total)</code> — ao criar dívida recorrente	Por causa da forma como o JavaScript representa números decimais, uma dívida de 1000 CVE em 3 parcelas de 333,33 é rejeitada ($333.33 \times 3 = 999.99$, não 1000), mesmo sendo um plano de pagamento perfeitamente válido. O utilizador recebe um erro incorreto e não consegue registar a dívida como pretendia.	Alta
"Hoje" é sempre calculado em UTC, não no fuso horário do utilizador.	<code>new Date().toISOString().slice(0,10)</code> , usado em dezenas de sítios	Cabo Verde está em UTC-1. Perto da meia-noite (entre 23h e 00h locais), a aplicação já considera que é "o dia seguinte", o que pode fazer um evento aparecer/desaparecer da	Média

Problema	Onde	Impacto / Cenário	Severidade
		lista de "próximos eventos" ou dos alertas de urgência um dia antes ou depois do esperado.	
O "Crescimento de Investimento" mostrado não é rentabilidade real, é apenas o total depositado.	cálculo de <code>growthRate</code> em <code>updateIndicatorCards()</code> , e o gráfico de crescimento acumulado	A taxa de juro introduzida por investimento (<code>interestRate</code>) fica guardada mas nunca entra em nenhum cálculo — não há juro composto aplicado. A "taxa de crescimento" mostrada é só a variação do total depositado ano a ano, o que pode confundir-se facilmente com retorno/rentabilidade real do investimento.	Média
Remover "todas as ocorrências" de uma série recorrente identifica a série pelo título (e, para dívidas, também pelo valor).	<code>removeAllOccurrences()</code>	Duas séries diferentes com o mesmo título (ex.: duas assinaturas de streaming chamadas "Subscrição") e do mesmo tipo podem ser apagadas em conjunto por engano, mesmo sendo eventos distintos — porque não existe um identificador de "série" ligando as ocorrências entre si.	Média

5. O Problema de Fundo: o Modelo de Dados

Este é o ponto mais importante de toda a auditoria, porque é a raiz de vários dos problemas acima e vai condicionar tudo o que vier a seguir — incluindo o agente de IA.

Hoje, cada evento tem um único campo "type" que pode ser: income, debt, essencial, leisure, savings, investment, emergency, others ou fixed. O problema é que este campo mistura três ideias que são conceptualmente diferentes:

- Natureza do movimento — é dinheiro a entrar (renda) ou a sair (despesa)?
- "Onde" o dinheiro está — poupança, investimento e cofre de emergência não são categorias de despesa, são contas/reservas com um saldo próprio que evolui ao longo do tempo.
- Categoria da despesa — essencial, lazer, despesas fixas, dívida e "outros" são, de facto, categorias de gasto.

Ao tratar estas três coisas como se fossen a mesma, aparecem consequências práticas: o gráfico de "Distribuição" soma na mesma pizza dinheiro gasto (lazer) com dinheiro guardado (poupança) e

dinheiro investido, o que não responde de forma limpa à pergunta "para onde foi o meu dinheiro este mês"; só é possível ter uma meta de poupança e uma meta de investimento globais, porque a meta está associada ao tipo e não a uma conta/objetivo específico — não há como ter, ao mesmo tempo, "poupança para férias" e "poupança para entrada de casa" com metas diferentes; e a distinção entre "essencial" e "despesas fixas" é ambígua mesmo para o próprio utilizador (renda de casa é as duas coisas?), o que ao fim de alguns meses tende a gerar categorização inconsistente.

A recomendação (detalhada na secção 8) é separar isto em quatro entidades: Contas (carteira, poupança, investimento, cofre de emergência — cada uma com saldo próprio), Transações (entradas e saídas, com uma categoria de gasto e ligadas a uma conta), Dívidas (com capital, juro, plano de pagamento e histórico) e Metas (valor-alvo e prazo, associadas a uma conta específica, permitindo várias metas em paralelo).

6. Lacunas Funcionais para Uso Real

Falhas de funcionalidade que vão ser sentidas assim que o uso deixar de ser só um teste:

- Não existe uma vista de "todas as transações" com pesquisa e filtros — só os 10 próximos eventos ficam visíveis na lista principal; não há como consultar o histórico completo dentro da aplicação.
- Não é possível ter mais do que uma conta corrente, cartão de crédito ou conta bancária — tudo cai num único "saldo atual".
- Não há suporte a múltiplas metas de poupança/investimento em paralelo (ver secção 5).
- Não há forma de importar extratos bancários (CSV/OFX) — todo o lançamento é manual, o que é a maior barreira ao uso contínuo de qualquer gestor financeiro pessoal.
- Moeda fixa em CVE, sem possibilidade de configurar outra — impede vender o produto fora de mercados que usam Escudo Cabo-verdiano sem alterar código.
- Idioma fixo em português europeu/cabo-verdiano no próprio código (strings escritas diretamente no HTML/JS) — dificulta internacionalização se o plano for vender a outros mercados.
- Sem tema claro/escuro alternável — o tema escuro está bem feito, mas fica sem alternativa.
- Os campos do formulário "Adicionar Evento" não são limpos automaticamente depois de guardar, obrigando a apagar tudo manualmente antes do lançamento seguinte.

7. Dívida Técnica

Questões de engenharia que não se veem no ecrã, mas que tornam o projeto difícil de manter e de evoluir com confiança:

- Ficheiro único com mais de 2000 linhas, misturando HTML, CSS e ~1000 linhas de JavaScript — difícil de manter, testar ou dividir entre várias pessoas.
- Sem qualquer teste automatizado — não há forma de garantir que uma alteração futura não voltou a partir o cálculo de saldo, por exemplo.
- Sem ferramenta de build, sem TypeScript, sem linter — erros de tipo (como somar texto com número) só aparecem em produção, ao vivo.
- Todas as bibliotecas (Bootstrap, FullCalendar, Chart.js, Lucide) são carregadas de um CDN externo, sem verificação de integridade (SRI) — se o CDN falhar ou for comprometido, a aplicação para de funcionar ou fica vulnerável.

- Números guardados como float (vírgula flutuante), o padrão habitual para dinheiro em código são valores inteiros (cêntimos) ou uma biblioteca decimal, para evitar erros de arredondamento acumulados.

8. Roadmap Proposto

Proponho quatro fases. Cada uma entrega valor por si só — não é preciso comprometer-se com o produto SaaS nem com o agente de IA para já; pode parar depois da Fase 1 ou da Fase 2 e já ter um gestor financeiro pessoal muito mais sólido.

Fase 1 — Estabilizar o que já existe (dias, não semanas)

Objetivo: parar de arriscar perder dados e corrigir os bugs de cálculo, sem mudar de tecnologia nem de arquitetura.

1. Adicionar exportação/importação de um ficheiro de segurança (JSON) — com um clique, salvar tudo para um ficheiro e, com outro, restaurar. Isto sozinho já elimina o maior risco atual (perda total de dados).
2. Corrigir os 5 bugs de cálculo da secção 4 (cofre de emergência, validação de parcelas, fuso horário, taxa de crescimento, remoção de séries).
3. Escapar corretamente qualquer texto do utilizador antes de o inserir na página (resolver o XSS).
4. Adicionar tratamento de erro à leitura do localStorage, para a aplicação nunca mais "morrer" por dados corrompidos.
5. Passar a usar identificadores verdadeiramente únicos (crypto.randomUUID()) para cada evento.
6. Pequenos ganhos de usabilidade: limpar o formulário depois de guardar, mostrar todo o histórico com pesquisa, alternância de tema claro/escuro.

Fase 2 — Arquitetura de produto real

Objetivo: sair do "só funciona neste browser" para uma aplicação com conta de utilizador, dados seguros na nuvem e um modelo de dados correto.

- Redesenhar o modelo de dados nas quatro entidades descritas na secção 5: Contas, Transações, Dívidas e Metas.
- Escolher uma base para autenticação e base de dados — por exemplo Supabase ou Firebase (mais rápido a montar, menos código de backend) ou uma stack própria em Node.js/PostgreSQL (mais controlo, mais trabalho). Para um projeto a solo com o objetivo de eventualmente vender, uma plataforma tipo Supabase costuma ser o equilíbrio certo entre velocidade e controlo.
- Login por email/palavra-passe e, idealmente, login social (Google) para reduzir fricção.
- Reescrever o frontend de forma modular (por exemplo com Vite + um framework como React, Vue ou Svelte, ou mesmo JavaScript modular sem framework) em vez de um único ficheiro HTML — mais fácil de manter e de testar.
- Guardar valores monetários como inteiros (cêntimos) e tratar datas sempre no fuso horário local do utilizador, nunca em UTC direto.
- Sincronização entre dispositivos passa a ser automática, porque os dados já não vivem só no browser.

Fase 3 — Preparar para vender (SaaS)

Objetivo: transformar o produto pessoal num serviço que outras pessoas possam subscrever com confiança.

- Suporte multi-utilizador com isolamento de dados entre contas (multi-tenancy).
- Faturação e subscrições (por exemplo com Stripe), com plano gratuito limitado e plano pago.
- Internacionalização: moeda configurável, idioma configurável (não hardcoded), formatação de números por região.
- Conformidade básica: política de privacidade, termos de uso, e um mecanismo de exportação/eliminação de dados do utilizador (importante em praticamente qualquer mercado, pela sensibilidade de dados financeiros).
- Interface responsiva testada a sério em telemóvel (é onde a maioria das pessoas vai querer lançar uma despesa no momento em que ela acontece).
- Importação de extratos bancários (CSV, e eventualmente ligação direta a bancos via open banking, se o mercado-alvo suportar) — reduz drasticamente a fricção de manter os dados atualizados, que é a razão número um pela qual gestores financeiros pessoais são abandonados.

Fase 4 — Agente de IA financeiro

Esta fase só faz sentido depois da Fase 2, porque um agente de IA é tão bom quanto os dados que consegue "ver". Com o modelo de dados ainda misturado (secção 5), qualquer resposta da IA sobre "posso gastar X" ou "devo investir ou pagar dívida" estaria a raciocinar sobre números pouco fiáveis.

- Com Contas, Transações, Dívidas e Metas bem separadas, um agente (por exemplo usando a API da Claude com acesso a estas tabelas através de "tools"/funções) pode responder a perguntas como: "Posso gastar 15.000 CVE em lazer este mês sem comprometer a minha meta de poupança?", "Qual dívida devo priorizar pagar primeiro?", "Detetei que os teus gastos em lazer subiram 30% este trimestre", ou "Com este ritmo, vais atingir a tua meta de emergência em 4 meses".
- O agente deve ser desenhado para dar sugestões informativas, não ordens — sempre deixando claro que não substitui aconselhamento financeiro profissional, especialmente em decisões de investimento.
- Tecnicamente, isto implica expor ao agente um conjunto de "ferramentas" (funções) que leem os dados do utilizador de forma estruturada e segura, e não dar-lhe acesso direto e sem filtros à base de dados inteira.

9. Próximos Passos Recomendados

Sugiro começar pela Fase 1: é rápida, elimina o maior risco (perda de dados) e corrige os bugs que já hoje distorcem os números que você vê. Depois disso, mesmo que decida não avançar já para SaaS ou IA, terá um gestor financeiro pessoal em que pode confiar no dia a dia.

Posso avançar já com a implementação da Fase 1 diretamente sobre este ficheiro (mantendo a mesma aparência e sem exigir nenhuma conta ou serviço externo), e deixar as Fases 2 a 4 planeadas em detalhe para quando fizer sentido para si avançar. Diga-me se quer que comece por aí.